Integración de Sistemas

Semana 9. Pruebas de integración y contract testing

Sistemas de Información — PUCE
Docente: Damián Nicolalde

Semana 9. Pruebas de integración y contract testing

Objetivo: Comprender la pirámide de pruebas (unitarias, integración, E2E) y aplicarla al ecosistema de microservicios construido durante el curso; dominar el contract testing consumer-driven con Pact como mecanismo para detectar breaking changes entre proveedor y consumidor antes de desplegar; distinguir los distintos tipos de test doubles (mocking) y su rol en aislar pruebas; e integrar todo lo anterior en un pipeline de CI/CD.

Resultado de aprendizaje asociado: Implementar un ecosistema de microservicios expuestos mediante APIs REST y gestionados con API Gateway y un bus de eventos, incorporando contratos, pruebas, observabilidad y seguridad para asegurar escalabilidad y desempeño.

Nota de continuidad de stack: el sílabo original de esta semana menciona xUnit (framework de .NET). Como la práctica del curso migró a NestJS/JavaScript, esta semana usa Jest — el framework nativo que 'nest new' configura por defecto — como equivalente directo dentro del ecosistema Node.js.

Introducción

Durante ocho semanas has construido un ecosistema creciente: una API REST de Productos con CRUD completo (Semanas 1-2), un servicio gRPC (Semana 3), un servidor GraphQL (Semana 4), una integración SOA con mensajería (Semana 5), una coreografía de eventos con compensaciones (Semana 6), una disciplina de contratos formales con JSON Schema (Semana 7) y un API Gateway centralizando el acceso (Semana 8). En ningún momento, sin embargo, te has hecho la pregunta que separa a un sistema que "parece funcionar" de un sistema del que puedes estar razonablemente seguro: ¿cómo sabes, de forma automática y repetible, que sigue funcionando después de cada cambio?

Sin pruebas automatizadas, la única forma de responder esa pregunta es probar manualmente cada endpoint después de cada cambio — un proceso que no escala, que un humano cansado hace mal, y que se vuelve prácticamente imposible en cuanto el sistema tiene más de un microservicio hablando con otro. Esta semana introduce la disciplina de pruebas automatizadas aplicada específicamente al problema de la integración: no solo "¿funciona mi función?" (eso lo resuelve una prueba unitaria), sino "¿siguen siendo compatibles mi proveedor y mi consumidor después de que uno de los dos cambió?" — la pregunta central del contract testing, que conecta directamente con la disciplina de contratos y backward compatibility que estudiaste en la Semana 7.

La experiencia de laboratorio de hoy trabaja sobre nestjs-productos-api (Semanas 1-2): escribirás pruebas unitarias, de integración y E2E con Jest, y luego construirás tu primer par consumidor-proveedor con Pact, provocando deliberadamente un breaking change para ver, en carne propia, cómo el contract testing lo detecta antes de que llegue a producción.

Antes de entrar en materia, conviene internalizar una idea que motiva toda la disciplina de esta semana: el costo de corregir un defecto crece dramáticamente cuanto más tarde se detecta en

el ciclo de vida del software. Un error detectado por una prueba unitaria mientras se escribe el código cuesta, en la práctica, segundos de atención del propio desarrollador. Ese mismo error, si escapa a producción y lo reporta un usuario final, puede costar horas de investigación, una corrección urgente bajo presión, un despliegue de emergencia, y en el peor caso datos corruptos o transacciones de negocio perdidas que ya no se pueden deshacer limpiamente. Este principio, documentado desde los años 70 (Barry Boehm) hasta estudios recientes de la industria, es la razón económica de fondo por la que invertir tiempo en pruebas automatizadas resulta, medido en el tiempo, más barato que no hacerlo.

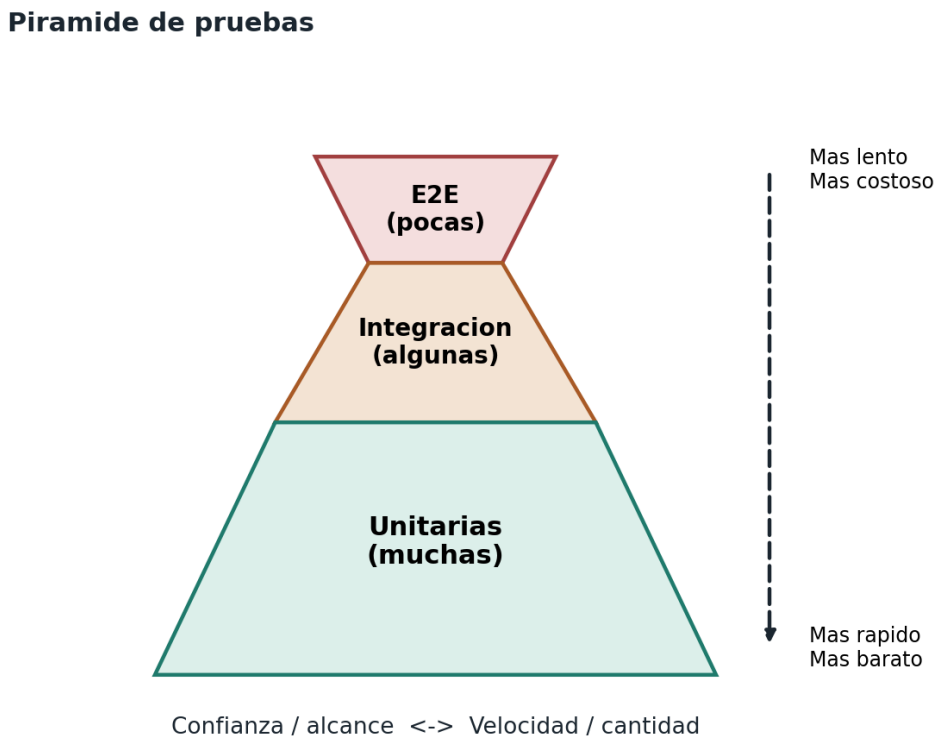
9.1. Pruebas unitarias, integración y E2E

La pirámide de pruebas

La pirámide de pruebas es un modelo mental, popularizado por Mike Cohn, que organiza los distintos tipos de prueba automatizada según tres ejes que se mueven en direcciones opuestas a medida que subes de nivel: costo de escribir y mantener, velocidad de ejecución, y alcance/confianza que otorgan sobre el comportamiento real del sistema.

- Base — Pruebas unitarias: verifican una única unidad de código de forma completamente aislada de sus dependencias externas. Son las más rápidas y baratas, pero las que menos confianza aportan sobre el sistema completo.
- Medio — Pruebas de integración: verifican que varias unidades reales colaboran correctamente entre sí, o que un componente se integra correctamente con una dependencia externa real. Detectan errores en el "pegamento" entre piezas.
- Cima — Pruebas E2E: verifican el sistema completo desde la perspectiva de un cliente externo real, golpeando la API HTTP de principio a fin. Las más lentas, frágiles y costosas de mantener, pero las que más se parecen a la experiencia real del usuario.

Diagrama 1. Pirámide de pruebas: unitarias en la base (muchas, rápidas, baratas), integración en el medio, E2E en la cima (pocas, lentas, costosas, alto alcance).



La recomendación clásica es tener muchas pruebas unitarias, algunas de integración, y pocas E2E. Un error extremadamente común en equipos que recién adoptan pruebas automatizadas es invertir la pirámide: escriben decenas de pruebas E2E lentas y frágiles porque "prueban todo de una vez", y casi ninguna unitaria, resultando en una suite que tarda 40 minutos y falla por razones ajenas al cambio real.

Pruebas unitarias en NestJS con Jest

NestJS genera automáticamente, al crear cada recurso, un archivo *.spec.ts ya configurado para usar Jest. Una prueba unitaria del ProductosService (Semanas 1-2) mockea por completo el almacenamiento subyacente:

```
describe("ProductosService", () => {
  let service: ProductosService;

  beforeEach(async () => {
    const module = await Test.createTestingModule({
      providers: [ProductosService],
    }).compile();
    service = module.get(ProductosService);
  });
```

```
  it("debe listar los 3 productos semilla", () => {
    expect(service.listar()).toHaveLength(3);
  });

  it("debe lanzar NotFoundException si el id no existe", () => {
    expect(() => service.obtenerPorId(999)).toThrow(NotFoundException);
  });
});
```

Cuando el servicio depende de una colaboración externa, la prueba unitaria reemplaza esa dependencia por un doble controlado con jest.fn(), de forma que la prueba nunca toca una base de datos real.

Pruebas de integración: sin mocks, con módulos reales

Una prueba de integración sube el nivel: en vez de instanciar el servicio aislado, levanta el TestingModule completo (controller + service reales, sin mocks) y verifica que colaboran correctamente. La diferencia parece sutil en un ejemplo pequeño, pero se vuelve crítica en sistemas reales: una prueba de integración detecta errores de "cableado" (un provider mal registrado, una inyección de dependencias mal configurada) que una prueba unitaria, al mockear todo, jamás podría ver.

Pruebas E2E con supertest

Las pruebas E2E van un paso más allá: levantan la aplicación HTTP completa en memoria y le envían peticiones HTTP reales, usando la librería supertest:

```
it("GET /productos debe devolver 200 y un array", () => {
  return request(app.getHttpServer())
    .get("/productos")
    .expect(200)
    .expect(res => expect(Array.isArray(res.body)).toBe(true));
});

it("POST /productos con datos invalidos debe devolver 400", () => {
  return request(app.getHttpServer())
    .post("/productos")
    .send({ nombre: "" })
    .expect(400);
});
```

Esta prueba ejercita, en un solo golpe, el ValidationPipe global, el routing HTTP real, el controller y el service — exactamente el mismo camino que recorrería una petición real, incluyendo el CRUD completo de la Semana 2.

Ejercicio resuelto 9.1 — Clasificar el nivel de una prueba

Enunciado: Clasifica cada prueba como unitaria, de integración o E2E: (a) verifica que calcularDescuento(100, 0.1) devuelve 90, sin tocar ninguna clase de NestJS; (b) levanta TestingModule con el ProductosService real y un repositorio mockeado; (c) hace POST /productos a la app HTTP completa y verifica el código de estado.

Solución: (a) Unitaria — una única función pura. (b) Integración — el service es real, el repositorio sigue mockeado (integración parcial). (c) E2E — atraviesa toda la pila HTTP real.

Snapshot testing con Jest: cuándo ayuda y cuándo estorba

Jest ofrece snapshot testing: en vez de escribir manualmente la aserción esperada, la primera vez que corre la prueba Jest guarda automáticamente el resultado actual en un archivo __snapshots__/, y en ejecuciones futuras compara el resultado nuevo contra esa fotografía guardada.

```
it("la respuesta de un producto debe mantener su forma", () => {
  const producto = service.obtenerPorId(1);
  expect(producto).toMatchSnapshot();
});
```

Es útil para verificar que estructuras de datos complejas no cambian de forma inesperada, sin escribir a mano una aserción exhaustiva. El riesgo: si un cambio legítimo modifica la forma de la respuesta, la prueba falla, y la forma 'fácil' de arreglarla —ejecutar jest --updateSnapshot sin revisar qué cambió— puede volverse un hábito peligroso, perdiendo el valor de la prueba como red de seguridad.

La recomendación práctica es reservar el snapshot testing para estructuras grandes o repetitivas, y siempre revisar conscientemente el diff antes de actualizar un snapshot — nunca como reflejo automático ante un fallo.

Taller en grupo 9.1 — Diseñar la pirámide con IA

En parejas, tomen el CRUD de nestjs-productos-api (Semana 2) y pidan a la IA: "Dado este controller y este service, propón qué pruebas unitarias, de integración y E2E escribirías para cada endpoint, y justifica el nivel de la pirámide de cada una." Verificación obligatoria: cuenten cuántas pruebas propuso en cada nivel — si el resultado no se parece a una pirámide, reequilibren ustedes mismos antes de implementarlo.

Test-Driven Development (TDD) y su relación con la pirámide

Un debate frecuente es si las pruebas deben escribirse antes o después del código de producción. Test-Driven Development (TDD) propone un ciclo disciplinado conocido como rojo-verde-refactor: primero se escribe una prueba que falla (rojo, porque el código aún no existe), luego se escribe el código mínimo para que pase (verde), y finalmente se mejora el diseño sin romper la prueba (refactor).

Es importante distinguir dos preguntas independientes: "¿en qué orden escribo pruebas y código?" (responde TDD) y "¿qué tipos de prueba necesito?" (responde la pirámide). TDD es una disciplina de flujo de trabajo que puede aplicarse en cualquier nivel de la pirámide — es perfectamente posible, y recomendable, hacer TDD a nivel unitario. La ventaja pedagógica de TDD es que obliga a pensar primero en el comportamiento observable esperado antes que en la implementación interna, produciendo interfaces más limpias y menos acopladas a detalles que cambiarán con el tiempo.

Jest, xUnit, JUnit, pytest: el mismo concepto en distintos ecosistemas

Vale la pena situar a Jest dentro de un panorama más amplio: la disciplina de esta semana es transferible a cualquier lenguaje, no solo a JavaScript/TypeScript.

Tabla 0b. Frameworks de pruebas equivalentes por ecosistema

Ecosistema	Framework típico	Runner
Node.js / TypeScript (este curso)	Jest (o Vitest)	npm test / jest
.NET / C# (sílabo original)	xUnit (o NUnit, MSTest)	dotnet test
Java	JUnit 5	mvn test / gradle test
Python	pytest (o unittest)	pytest
Go	paquete testing de la librería estándar	go test

Todos comparten la misma estructura conceptual —una función de prueba, una aserción sobre un resultado esperado, un setup/teardown— aunque cada uno tenga su propia sintaxis. Si en tu futuro profesional trabajas con un stack distinto, la pirámide, el contract testing y la taxonomía de test doubles siguen aplicando exactamente igual; solo cambia el nombre del framework.

Cobertura de código: qué mide y qué no mide

Una métrica que acompaña a una suite de pruebas es la cobertura de código: el porcentaje de líneas, ramas o funciones que se ejecutan al menos una vez durante la suite. Jest la calcula con la bandera --coverage.

La cobertura es útil pero traicionera si se malinterpreta: un 100% significa que cada línea se ejecutó al menos una vez, NO que se probó con todos los valores límite relevantes, ni que las aserciones verifican el comportamiento correcto. Es posible ejecutar una línea sin verificar nada sobre su resultado, inflando la cobertura sin aportar confianza real.

Tabla 0c. Lo que la cobertura SÍ mide vs. lo que NO mide

Sí mide	No mide
Qué líneas/ramas se ejecutaron al menos una vez	Si las aserciones verifican el comportamiento correcto
Código muerto que ninguna prueba toca	Casos límite no contemplados (negativos, nulos, vacíos)
Un umbral objetivo para gates de CI	La calidad del diseño del código bajo prueba

La recomendación práctica es usar la cobertura como señal de alerta, no como objetivo a maximizar por sí mismo — perseguir 100% sin cuestionar la calidad de las aserciones es una forma sutil de teatro de calidad.

Ejercicio resuelto 9.1b — Aplicar el ciclo rojo-verde-refactor

Enunciado: Quieres agregar a ProductosService un método buscarPorNombre(texto) que devuelva los productos cuyo nombre contenga el texto dado (sin distinguir mayúsculas/minúsculas). Describe los tres pasos del ciclo TDD.

Solución: Rojo: escribir una prueba que busque 'teclado' esperando encontrarlo aunque se busque 'TECLADO'; el método aún no existe, la prueba falla. Verde: implementar la versión más simple que la satisfaga (un filter con toLowerCase en ambos lados). Refactor: limpiar el código (extraer la normalización a una función auxiliar) verificando que la prueba sigue en verde.

9.2. Contract testing con Pact y Postman

El problema que las pruebas E2E tradicionales no resuelven

Imagina que tu ecosistema creciera a 10 microservicios reales, cada uno con su propio equipo. Probar de forma E2E que un cambio no rompe la integración entre dos de ellos exigiría levantar los 10 microservicios completos con sus bases de datos y colas — un proceso lento, frágil y costoso de mantener en CI. El contract testing ataca el problema desde otro ángulo: cada lado prueba, por separado, que cumple un contrato explícito y compartido, sin necesidad de levantar ambos servicios simultáneamente.

Pact: consumer-driven contracts

Pact es el framework de contract testing más usado en el ecosistema JavaScript/Node. Su filosofía es consumer-driven: es el consumidor quien define, mediante una prueba normal en su propio código, exactamente qué espera recibir del proveedor. Al ejecutarla, Pact genera automáticamente un archivo de contrato (pact.json) que se comparte con el proveedor a través de un Pact Broker, y el proveedor ejecuta una verificación que reproduce las interacciones del pact.json contra su implementación real.

Diagrama 2. Flujo de contract testing consumer-driven: el consumidor define su expectativa, Pact genera pact.json, el Pact Broker lo centraliza y versiona, y el proveedor descarga y verifica.



Postman/Newman como alternativa más ligera

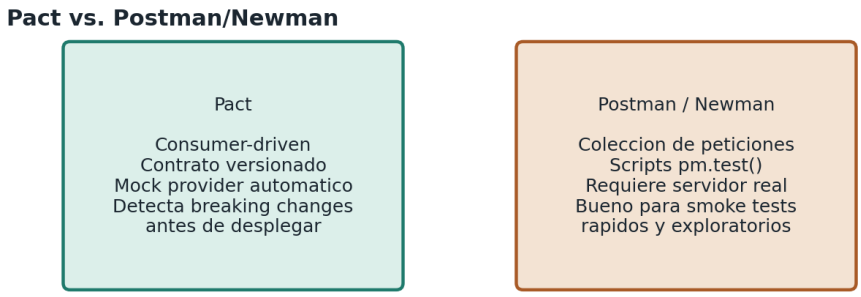
Postman también puede usarse como una forma más ligera de contract testing: se escriben scripts pm.test() dentro de cada petición de una colección, ejecutada en CI mediante Newman. La diferencia clave es que Postman/Newman prueba contra un servidor real ya desplegado, por lo que es más simple de adoptar pero ofrece garantías más débiles que Pact.

```
pm.test("Status code es 200", () => pm.response.to.have.status(200));
pm.test("La respuesta tiene el campo nombre", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("nombre");
});
```

Tabla 1. Pact vs. Postman/Newman

Aspecto	Pact	Postman/Newman
Filosofía	Consumer-driven, contrato versionado	Colección de pruebas contra servidor real
Requiere servidor real corriendo	No (mock provider)	Sí
Detecta breaking changes antes de desplegar	Sí, sistemáticamente	Solo si se ejecuta antes del despliegue
Curva de aprendizaje	Más alta	Más baja
Mejor para	Ecosistemas con muchos microservicios	Smoke tests exploratorios

Diagrama 6. Comparación visual entre Pact (consumer-driven, contrato versionado, detecta breaking changes) y Postman/Newman (colección de peticiones, requiere servidor real, ideal para smoke tests).



Contract testing bidireccional: consumidor y proveedor a la vez

Un matiz que confunde a quien recién adopta Pact es que, en un ecosistema real, un mismo servicio casi nunca ocupa un solo rol de forma permanente. nestjs-bff-productos (Semana 8) actúa como consumidor frente a nestjs-productos-api y frente a nestjs-productos-graphql, pero podría convertirse en proveedor el día que una app móvil externa consuma directamente sus endpoints agregados.

Esto significa que un mismo proyecto NestJS puede tener, simultáneamente, pruebas de consumidor (generan pact.json describiendo lo que espera de sus propias dependencias) y pruebas de verificación de proveedor (confirman que cumple los contratos que sus propios consumidores han publicado sobre él). El rol de 'consumidor' o 'proveedor' no es una propiedad fija de un servicio, sino de cada relación específica entre dos servicios.

Ejercicio resuelto 9.2 — Elegir la herramienta correcta

Enunciado: Tu equipo tiene 3 microservicios mantenidos por 3 sub-equipos distintos y quiere garantizar que ningún cambio de un proveedor rompa a sus consumidores sin que nadie se entere hasta producción. ¿Pact o Postman/Newman?

Solución: Pact — el escenario (múltiples equipos, necesidad de detectar breaking changes antes de desplegar) es exactamente el problema para el que Pact fue diseñado.

Taller en grupo 9.2 — Diseñar un contrato con IA

En parejas, describan a la IA un endpoint propio (ej. GET /productos/:id) y pidan: "Genera el esqueleto de una prueba de consumidor Pact en Jest/TypeScript para este endpoint, incluyendo la interacción esperada." Verificación obligatoria: ejecuten la prueba generada — si Pact no logra generar el pact.json sin errores, el ejercicio no está completo.

Origen histórico: por qué "consumer-driven"

El término consumer-driven contract fue acuñado por Ian Robinson en 2006, pero la herramienta que lo popularizó masivamente, Pact, nació en REA Group (Australia) hacia 2013,

cuando su equipo enfrentaba decenas de microservicios propios, mantenidos por equipos distintos, que se rompían entre sí con frecuencia cada vez que un proveedor cambiaba algo que "creía" no afectaba a nadie. Invertir la dirección tradicional de las pruebas de contrato — dejando que sea el consumidor quien defina explícitamente sus expectativas— resultó tan efectivo que el proyecto se abrió como open source y hoy tiene implementaciones en más de una decena de lenguajes.

Vale la pena contrastarlo con el enfoque anterior, a veces llamado provider-driven: el proveedor publica su documentación (un OpenAPI, por ejemplo) y espera que los consumidores se adapten, sin garantía de que la implementación real coincida exactamente con esa documentación, ni de qué parte de ella usa realmente cada consumidor. El enfoque consumer-driven deriva el contrato de un uso real y verificado, no de una intención documentada que puede desactualizarse.

Ejercicio resuelto 9.2b — Provider-driven vs. consumer-driven

Enunciado: Un proveedor elimina un campo descripcionLarga de su respuesta porque "nadie debería usarlo" según su documentación OpenAPI. Una app móvil que sí lo usaba deja de funcionar. ¿Qué enfoque habría prevenido esto?

Solución: Un enfoque consumer-driven con Pact: si la app móvil tuviera un test de consumidor declarando explícitamente su dependencia de ese campo, la verificación del proveedor habría fallado antes del despliegue.

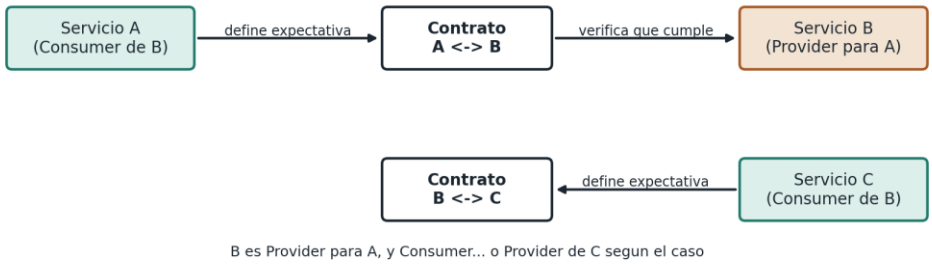
9.3. Provider–consumer testing

Roles y responsabilidades

- Consumidor (consumer): el sistema que llama a otro. Define, mediante su propia prueba, qué espera recibir.
- Proveedor (provider): el sistema que responde. Debe verificar que su implementación real satisface todos los contratos publicados por sus consumidores.

Un mismo microservicio puede ser consumidor en una relación y proveedor en otra simultáneamente — nestjs-bff-productos (Semana 8) es consumidor de nestjs-productos-api, pero podría ser proveedor de una app móvil.

Diagrama 3. Roles Consumer y Provider con el contrato en el medio: un mismo servicio (B) puede ser Provider para A y Consumer de otro servicio C.



El flujo completo paso a paso

1. El equipo consumidor escribe una prueba usando el mock provider de Pact, describiendo la interacción esperada.

2. Al correr la prueba, Pact genera pact.json y lo publica en un Pact Broker compartido con la versión del consumidor.

3. El equipo proveedor descarga el pact.json relevante y ejecuta la verificación del proveedor: Pact reproduce cada interacción contra el servidor real y compara la respuesta.

4. Si la verificación falla, el proveedor sabe, antes de desplegar, que rompería a ese consumidor específico.

Por qué esto detecta breaking changes antes de desplegar

Esta es la conexión más directa con la Semana 7: un contrato Pact es una forma ejecutable de verificar backward compatibility. Si el proveedor renombra un campo de la respuesta (nombre → nombreProducto, el mismo tipo de cambio estudiado como breaking change en la Semana 7), la verificación falla inmediatamente, señalando qué campo esperado no apareció — mucho antes de que ese cambio llegue a producción.

El Pact Broker como fuente de verdad centralizada

- Almacena todas las versiones de todos los contratos entre todos los pares consumidor-proveedor.
- Calcula automáticamente una matriz de compatibilidad entre versiones.
- Expone el concepto 'can-i-deploy': pregunta si una nueva versión del proveedor satisface a todas las versiones de consumidores actualmente en producción.

El Pact Broker en detalle: can-i-deploy paso a paso

Para hacer tangible el concepto de can-i-deploy, vale la pena ver cómo se invoca desde la línea de comandos, típicamente como último paso de un pipeline antes de desplegar:

```
pact-broker can-i-deploy \
  --pacticipant ProductosApi \
```

```
  --version $GIT_SHA \
  --to-environment produccion
```

Este comando le pregunta al Pact Broker: "si despliego esta versión de ProductosApi a producción ahora, ¿seguiría satisfaciendo los contratos de todos los consumidores ya desplegados?". El Broker responde con un código de salida 0 (seguro desplegar) o distinto de cero (rompería a algún consumidor), basándose en verificaciones ya registradas, sin volver a ejecutar ninguna prueba en ese momento.

Esta capacidad escala el contract testing más allá de dos servicios: en un ecosistema con 15 consumidores de un mismo proveedor, verificar manualmente que un despliegue no rompe a ninguno sería propenso a errores; can-i-deploy convierte esa pregunta en un único comando determinista, ejecutado como gate de calidad (conexión directa con 9.5).

Ejercicio resuelto 9.3 — Identificar quién rompió el contrato

Enunciado: La verificación del proveedor falla con: "expected field precio to be a number, got string". ¿Quién cambió qué?

Solución: El proveedor cambió el tipo del campo precio en su respuesta real, sin que el consumidor haya cambiado nada. Debe revertir el cambio o coordinarse explícitamente con el consumidor.

Ejemplo completo: contrato Pact para GET /productos/:id

Para consolidar el flujo completo, vale la pena ver ambos lados del contrato con código real, tal como los construirás en la práctica de hoy.

Lado consumidor (define la expectativa contra un mock provider levantado automáticamente por Pact):

```
const provider = new Pact({ consumer: "BffProductos", provider: "ProductosApi" });

describe("Contrato con ProductosApi", () => {
  beforeAll(() => provider.setup());
  afterEach(() => provider.verify());
  afterAll(() => provider.finalize());

  it("GET /productos/1 debe devolver nombre y precio", async () => {
    await provider.addInteraction({
      state: "el producto 1 existe",
      uponReceiving: "una peticion para el producto 1",
      withRequest: { method: "GET", path: "/productos/1" },
      willRespondWith: {
        status: 200,
        body: { id: 1, nombre: Matchers.like("Teclado mecanico"), precio:
Matchers.decimal() },
      },
    });
    const respuesta = await obtenerProducto(1);
    expect(respuesta.nombre).toBeDefined();
```

```
  });
});
```

Lado proveedor (verifica que la implementación real cumple ese mismo contrato):

```
const opts = {
  provider: "ProductosApi",
  providerBaseUrl: "http://localhost:3000",
  pactUrls: ["./pacts/bffproductos-productosapi.json"],
  stateHandlers: { "el producto 1 existe": () => Promise.resolve() },
};

new Verifier(opts).verifyProvider().then(() => console.log("Contrato verificado"));
```

Nota el uso de Matchers.like(...) y Matchers.decimal() en vez de valores exactos: el consumidor no debería exigir que el nombre sea literalmente "Teclado mecanico" para siempre (eso haría el contrato demasiado frágil), sino que declara la forma esperada — un principio que conecta con la distinción entre JSON Schema estructural y validación de valores exactos vista en la Semana 7.

Ejercicio resuelto 9.3b — Diseñar un matcher apropiado

Enunciado: Estás definiendo el contrato de GET /productos. ¿Deberías exigir exactamente 3 productos, o algo más flexible?

Solución: Algo más flexible — declarar que la respuesta es un array donde cada elemento cumple cierta forma, sin fijar la cantidad exacta. Fijar el número exacto acoplaría la prueba a un estado de datos específico que cambiará tan pronto se agregue un producto más, generando fallos que no reflejan ninguna incompatibilidad real.

Taller en grupo 9.3 — Simular una negociación de contrato

En equipos de 4, dos representan al consumidor y dos al proveedor. El consumidor pide a la IA el contrato que esperaría de un endpoint de detalle de pedido; el proveedor, sin ver esa conversación, describe la respuesta real de su endpoint. Verificación obligatoria: comparen ambos resultados y listen cualquier discrepancia — eso es exactamente lo que el contract testing existe para atrapar.

9.4. Mocking de servicios

Taxonomía de test doubles

El término genérico "mock" se usa con frecuencia de forma imprecisa. Gerard Meszaros (xUnit Test Patterns) distingue cinco tipos de test doubles:

- Dummy: se pasa solo para satisfacer una firma de método, nunca se usa realmente.
- Stub: devuelve respuestas predefinidas y fijas, sin lógica real.
- Spy: como un stub, pero además registra cómo fue llamado (cuántas veces, con qué argumentos).
- Mock (estricto): preprogramado con expectativas explícitas que fallan la prueba si no se cumplen.
- Fake: implementación funcional simplificada con lógica real, no apta para producción (ej. un array en memoria en vez de una base de datos).

Diagrama 4. Taxonomía de los cinco test doubles (dummy, stub, spy, mock, fake) ordenados de menor a mayor sofisticación de comportamiento.



Mocking de dependencias internas con Jest

```
const inventarioMock = { verificarStock: jest.fn().mockResolvedValue(true) };
// ...
expect(inventarioMock.verificarStock).toHaveBeenCalledWith(productoId, 3);
```

Mocking de llamadas HTTP externas

Cuando el sistema bajo prueba llama a un servicio HTTP externo real (ej. nestjs-bff-productos llamando a la API GraphQL), librerías como nock interceptan las llamadas salientes y devuelven respuestas simuladas:

```
import nock from "nock";

nock("http://localhost:3000")
  .get("/productos/1")
  .reply(200, { id: 1, nombre: "Teclado mecanico", precio: 45.90 });
```

Una alternativa más moderna, msw (Mock Service Worker), intercepta a nivel de la capa de red del propio runtime, siendo popular cuando el mismo conjunto de mocks se reutiliza en pruebas y en desarrollo local del frontend.

Por qué aislar del mundo exterior

El valor de aislar el sistema bajo prueba con dobles es doble: velocidad (sin round-trips de red reales) y determinismo (una prueba que depende de un servicio externo real puede fallar por razones ajenas al código bajo prueba). Una suite que falla intermitentemente sin que el código cambie (flaky tests) erosiona rápidamente la confianza del equipo, al punto de ignorar fallos reales asumiendo que "es flaky otra vez".

Errores comunes al adoptar mocking

- Mockear el sistema bajo prueba en vez de sus dependencias — si mockeas la propia unidad que se supone estás probando, la prueba no verifica nada real de ella.
- Dobles que quedan desactualizados respecto a la interfaz real: si el método real cambia su firma, un stub escrito a mano puede seguir 'funcionando' mientras el código real ya rompió el contrato — riesgo que Pact, al basarse en interacciones HTTP reales, no sufre igual.

- Confundir 'la prueba pasa' con 'el comportamiento es correcto': aserciones débiles pueden pasar siempre sin detectar ningún regreso real.
- No limpiar los mocks entre pruebas: olvidar jest.clearAllMocks() puede filtrar estado de una prueba a la siguiente, produciendo fallos intermitentes difíciles de diagnosticar.

Ejercicio resuelto 9.4 — Elegir el test double correcto

Enunciado: Necesitas probar que, cuando PagoService recibe InventarioReservado, llama exactamente una vez a procesarPago con el monto correcto. ¿Qué test double necesitas como mínimo?

Solución: Un spy — no basta con que el doble devuelva un valor fijo (eso sería solo un stub), necesitas verificar cómo fue llamado.

Tabla de decisión: qué test double usar

Tabla 2b. Qué test double usar según la necesidad

Necesitas...	Test double apropiado
Pasar un argumento requerido que la lógica ignora	Dummy
Que una dependencia devuelva siempre el mismo valor fijo	Stub
Verificar que un método fue llamado con ciertos argumentos	Spy
Que la prueba falle si una llamada esperada no ocurre	Mock (estricto)
Una versión simplificada pero funcional de una dependencia	Fake

Ejercicio resuelto 9.4c — Configurar y limpiar un spy correctamente

Enunciado: Escribe el esqueleto correcto de una prueba que verifique que NotificacionesService.enviar es llamado exactamente una vez al procesar un pedido exitoso, asegurando que el estado del spy no se filtre a la siguiente prueba.

Solución:

```
describe("PedidoService", () => {
  let notificacionesSpy;

  beforeEach(() => {
    notificacionesSpy = jest.spyOn(notificacionesService, "enviar")
      .mockResolvedValue(undefined);
  });

  afterEach(() => {
    jest.clearAllMocks(); // evita "contaminar" la siguiente prueba
  });
```

```
  it("debe notificar exactamente una vez tras un pedido exitoso", async () => {
    await pedidoService.procesar(pedidoValido);
    expect(notificacionesSpy).toHaveBeenCalledTimes(1);
  });
});
```

El detalle que suele pasarse por alto es el afterEach con jest.clearAllMocks(): sin él, el contador de llamadas del spy persistiría entre pruebas del mismo describe, y una segunda prueba podría reportar '2 llamadas' arrastrando la llamada de la prueba anterior.

Ejercicio resuelto 9.4b — Sobre-mockeo que oculta un bug real

Enunciado: Un compañero mockea por completo el ProductosService al probar el ProductosController, incluyendo crear, de forma que la prueba siempre pasa sin importar el payload. Semanas después, en producción, POST /productos no valida correctamente el precio negativo. ¿Qué falló?

Solución: El mock ocultó por completo la lógica de validación real — nunca se ejecutó código real de validación durante la prueba del controller. La solución no es eliminar el mock, sino complementarlo con una prueba del ProductosService real que sí ejercite la validación de precio negativo.

Taller en grupo 9.4 — Auditar mocks excesivos con IA

En parejas, tomen un archivo de prueba propio y pidan a la IA: "Revisa estas pruebas y señala si algún mock está ocultando un bug real." Verificación obligatoria: decidan explícitamente, para cada mock señalado, si están de acuerdo, y documenten el razonamiento.

9.5. Pruebas automáticas en CI/CD

Por qué las pruebas manuales no escalan

Ejecutar manualmente una suite antes de cada despliegue depende de la disciplina humana: alguien debe acordarse de hacerlo, completo, cada vez, sin excepción. Bajo presión de tiempo, ese "sin excepción" es exactamente lo primero que se sacrifica — y es cuando más se necesitaría la red de seguridad de las pruebas.

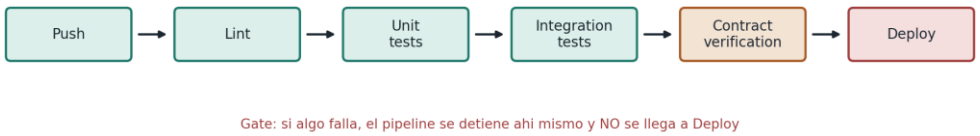
Estructura típica de un workflow de CI

```
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run test
      - run: npm run test:e2e
      - run: npm run test:pact:verify
```

Gates de calidad

Un gate de calidad es una regla que bloquea automáticamente una acción (fusionar un PR, desplegar) si no se cumple una condición mínima. GitHub y GitLab permiten configurar "required status checks": un Pull Request no puede fusionarse mientras el workflow de pruebas no reporte éxito.

Diagrama 5. Pipeline de CI/CD con gate de calidad: push, lint, pruebas unitarias, integración, verificación de contrato y despliegue; si algo falla, el pipeline se detiene antes de desplegar.



Un pipeline completo, extremo a extremo, con publicación y verificación de Pact

Para hacer completamente concreto lo estudiado, veamos un pipeline de GitHub Actions que integra las cinco secciones de esta semana en dos workflows separados y coordinados -uno para el consumidor, otro para el proveedor- comunicados a través del Pact Broker.

Workflow del consumidor (nestjs-bff-productos), que corre en cada push:

```
name: Consumer Pipeline
on: [push]
jobs:
  test-and-publish-pact:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run test
      - run: npm run test:pact
      - name: Publicar el contrato al Pact Broker
        run: |
          npx pact-broker publish ./pacts \
            --consumer-app-version=${{ github.sha }} \
            --broker-base-url=${{ secrets.PACT_BROKER_URL }}
```

Workflow del proveedor (nestjs-productos-api), que corre en cada push y cada vez que el Broker notifica un contrato nuevo relevante:

```
name: Provider Pipeline
on: [push, repository_dispatch]
jobs:
  test-and-verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run test
      - run: npm run start:test &
```

```
      - run: npm run test:pact:verify
      - name: Comprobar si es seguro desplegar
        run: |
          npx pact-broker can-i-deploy \
            --pacticipant ProductosApi \
            --version ${{ github.sha }} \
            --to-environment produccion
      - run: npm run deploy
```

Este par de workflows materializa, con herramientas reales, todo el flujo descrito en el Diagrama 2 y en la sección 9.3: el consumidor publica su expectativa apenas cambia su propio código, el proveedor se entera y verifica automáticamente que sigue cumpliendo esa expectativa, y ningún despliegue ocurre sin que can-i-deploy confirme que es seguro. En ningún momento el proveedor necesitó acceso al código fuente del consumidor, ni viceversa — toda la coordinación ocurre a través del contrato versionado en el Broker, exactamente el desacoplamiento entre equipos que el contract testing promete.

Dónde encaja la verificación de contrato en el pipeline

La verificación del proveedor suele correr como un job separado, después de las pruebas propias del proveedor, pero antes del despliegue. La funcionalidad 'can-i-deploy' del Pact Broker se invoca típicamente como el último gate antes del despliegue real, después de que todas las demás pruebas ya pasaron.

Ejercicio resuelto 9.5 — Diseñar el orden de un pipeline

Enunciado: Ordena estos pasos para detectar cada fallo lo antes posible: despliegue a producción, pruebas E2E, lint, pruebas unitarias, verificación de contrato.

Solución: lint → unitarias → E2E → verificación de contrato → despliegue. La regla de 'fail fast' es ordenar de más rápido/barato a más lento/costoso.

Ejercicio resuelto 9.5b — Diagnosticar un pipeline lento

Enunciado: El pipeline de tu equipo tarda 25 minutos, y la mayoría del tiempo lo consumen las pruebas E2E, que además fallan intermitentemente 1 de cada 10 veces sin que nadie haya cambiado el código. El equipo considera desactivarlas para 'ir más rápido'. ¿Es buena idea?

Solución: No, al menos no como primera respuesta — elimina la única capa que verifica el sistema completo desde la perspectiva de un cliente real. El diagnóstico correcto va en dos direcciones: investigar la causa de la intermitencia (flaky tests, 9.4) en vez de asumir que es normal; y revisar si el número de E2E es proporcional al criterio de 'pocas, solo los flujos más críticos' (9.1) — es posible que se hayan acumulado E2E para casos que deberían cubrirse con pruebas de integración, mucho más rápidas.

Taller en grupo 9.5 — Revisar un workflow de CI con IA

En parejas, pidan a la IA un workflow de GitHub Actions que corra lint, unitarias, E2E y verificación Pact en ese orden, deteniéndose en el primer fallo. Verificación obligatoria: revisen línea por línea el YAML contra la sintaxis oficial de GitHub Actions.

Más allá de la cobertura: una introducción a mutation testing

La sección 9.1 advirtió que la cobertura, por sí sola, no garantiza aserciones rigurosas. El mutation testing ataca ese punto ciego: una herramienta (como Stryker, para JS/TS) introduce automáticamente pequeños cambios deliberados ('mutantes') en el código de producción (ej. cambiar un > por un >=) y vuelve a correr la suite contra cada mutante.

Si la suite detecta el mutante (al menos una prueba falla), el mutante fue 'eliminado' — buena señal. Si ninguna prueba lo detecta, el mutante 'sobrevivió' — una señal de alerta concreta de una aserción débil, exactamente el tipo de hueco que la cobertura tradicional no revela, porque la línea mutada sí fue ejecutada, solo que ninguna aserción dependía realmente de su comportamiento exacto.

Es computacionalmente costoso, por lo que rara vez corre en cada push; es más común reservarlo para auditorías periódicas de módulos críticos. Da una vara de medir más honesta que '¿tengo 90% de cobertura?' para la pregunta que realmente importa: ¿mis pruebas detectarían un bug si apareciera?

Caso de estudio integrador: el pipeline completo de nestjs-productos-api

Cerramos la semana integrando las cinco secciones anteriores en un solo escenario, siguiendo el mismo patrón de cierre usado en las Semanas 7 y 8. Imagina que tu propio nestjs-productos-api va a incorporar, de una vez, toda la disciplina de pruebas de esta semana:

1. Pirámide (9.1): el equipo escribe primero pruebas unitarias exhaustivas del ProductosService, luego un número pequeño de pruebas de integración del módulo completo, y finalmente 3-4 pruebas E2E que cubren únicamente los flujos de negocio más críticos.

2. Mocking disciplinado (9.4): las pruebas unitarias usan stubs y spies para las dependencias internas; ninguna prueba mockea el propio ProductosService al probarlo a él mismo.

3. Contract testing (9.2-9.3): nestjs-bff-productos, como consumidor real, mantiene su propio test de consumidor Pact; nestjs-productos-api, como proveedor, descarga ese contrato y lo verifica antes de cada despliegue.

4. CI/CD con gate de calidad (9.5): un workflow ejecuta, en orden de fail fast, lint -> unitarias -> integración -> E2E -> verificación de contrato, y solo si las cinco etapas pasan se permite el despliegue.

El resultado es un sistema donde ningún cambio -ni siquiera uno aparentemente inocuo, como renombrar un campo de respuesta- puede llegar a producción sin que, en cuestión de minutos y sin intervención humana, algún nivel de esta red de seguridad lo detecte. Ninguna de estas capas sustituye a las demás: juntas conforman la disciplina de pruebas que separa un sistema que 'parece funcionar' de uno del que se puede estar razonablemente seguro.

Señales de que a tu proyecto le falta un tipo de prueba

Para cerrar, esta tabla resume síntomas observables en un proyecto real y qué nivel de la disciplina de pruebas probablemente falta o está débil:

Tabla 5b. Síntomas y su causa probable en la disciplina de pruebas

Síntoma observado	Probablemente falta...
Un cambio pequeño rompe otra parte del sistema sin que nadie lo note hasta producción	Pruebas unitarias con buena cobertura de casos límite
El controller compila y corre pero el endpoint devuelve 500 por un provider mal registrado	Pruebas de integración del módulo completo
Cada release manual encuentra un flujo de negocio roto que 'funcionaba la semana pasada'	Pruebas E2E de los flujos críticos
Un equipo despliega su microservicio y rompe silenciosamente a otro equipo que lo consumía	Contract testing (Pact) entre consumidor y proveedor
Las pruebas 'pasan siempre' pero los bugs igual llegan a producción	Aserciones débiles o mocks excesivos (revisar con mutation testing)
El pipeline tarda demasiado y el equipo empieza a saltárselo bajo presión	Reordenar el pipeline para fail-fast, o revisar el balance de la pirámide

Ninguna fila es exhaustiva por sí sola -un sistema real puede presentar varios síntomas a la vez-, pero sirve como primer diagnóstico rápido antes de decidir dónde invertir el siguiente esfuerzo de pruebas.

Resumen de la semana

Tabla 2. Resumen por sección

Sección	Idea central
9.1	La pirámide de pruebas equilibra costo, velocidad y confianza: muchas unitarias, algunas de integración, pocas E2E.
9.2	Pact permite contract testing consumer-driven sin levantar ambos servicios; Postman/Newman es más ligero y menos riguroso.
9.3	El flujo consumer → pact.json → Pact Broker → verificación del proveedor detecta breaking changes antes de desplegar.
9.4	Existen cinco test doubles (dummy, stub, spy, mock, fake); aislar del exterior da velocidad y determinismo.
9.5	Un pipeline de CI/CD con gates de calidad automatiza lo que la disciplina manual no garantiza de forma consistente.

Preguntas frecuentes

¿Necesito los 3 niveles de pruebas para cada endpoint? No en igual proporción — concentra el esfuerzo en unitarias, usa integración en los puntos de colaboración más riesgosos, reserva E2E para los flujos más críticos.

¿Pact reemplaza las pruebas E2E? No — verifica compatibilidad de contrato, no que el flujo de negocio completo tenga sentido de principio a fin.

¿Qué pasa si consumidor y proveedor son del mismo equipo? El valor de Pact es menor, pero sigue siendo útil como documentación ejecutable y red de seguridad ante cambios accidentales.

¿Cuántos mocks son demasiados? Cuando ya no queda claro qué comportamiento real se está verificando — señal de que el diseño tiene demasiadas dependencias.

¿Debo probar código que 'obviamente' funciona? Depende del costo de que falle silenciosamente. Una regla de negocio crítica sí merece prueba, sin importar cuán obvia parezca hoy — protege contra el cambio accidental de mañana.

¿El snapshot testing reemplaza a las aserciones tradicionales? No — es complementario para estructuras grandes y repetitivas. Para lógica puntual, una aserción explícita sigue siendo más clara de revisar en una Pull Request.

¿Vale la pena mutation testing en un proyecto pequeño? Probablemente no como práctica rutinaria, pero sí como ejercicio educativo puntual para descubrir aserciones débiles que la cobertura no revela.

Recursos de profundización

- Pactflow y can-i-deploy a escala real: explorar el servicio gestionado de Pact Broker y probar can-i-deploy en un escenario con más de dos servicios.
- Cobertura como diagnóstico, no como meta: generar un reporte con jest --coverage y usarlo para identificar las 3 líneas con menor cobertura, decidiendo conscientemente si merecen una prueba nueva.
- nock vs. msw en un mismo escenario: implementar el mismo mock HTTP con ambas librerías y documentar las diferencias de ergonomía.
- jest.mock() automático vs. manual: leer la documentación oficial y decidir cuál usarías para mockear un cliente HTTP externo.
- Un primer experimento de mutation testing: instalar Stryker sobre tu proyecto y revisar manualmente qué mutantes sobrevivieron y por qué.

Glosario de términos

- Pirámide de pruebas: modelo que organiza unitarias/integración/E2E según costo, velocidad y alcance.
- Prueba unitaria: verifica una unidad de código aislada de sus dependencias.
- Prueba de integración: verifica la colaboración real entre unidades o con una dependencia externa real.
- Prueba E2E: verifica el sistema completo desde la perspectiva de un cliente externo.
- Contract testing: verificación automatizada, por separado, de que consumidor y proveedor cumplen un contrato compartido.
- Consumer-driven contract: contrato definido por las expectativas del consumidor.
- Pact Broker: servidor centralizado que almacena, versiona y calcula compatibilidad entre contratos.
- Test double: objeto que sustituye a una dependencia real en una prueba.
- Flaky test: prueba que falla de forma intermitente sin que el código haya cambiado.
- Gate de calidad: regla automatizada que bloquea una acción si no se cumple una condición mínima.
- CI/CD: integración continua / despliegue continuo.
- Mutation testing: introduce cambios deliberados ('mutantes') en el código para verificar si la suite es lo suficientemente rigurosa como para detectarlos.
- Snapshot testing: técnica de Jest que compara el resultado actual contra una fotografía guardada previamente.
- can-i-deploy: funcionalidad del Pact Broker que responde si una versión de un servicio es segura de desplegar dados los contratos ya verificados.

Nota de cierre para el docente

Esta semana es, deliberadamente, la más 'invisible' del curso en términos de funcionalidad nueva construida: al final del laboratorio, nestjs-productos-api hace exactamente lo mismo que hacía al final de la Semana 2. Lo que cambia no es el comportamiento del sistema, sino la confianza demostrable que el equipo tiene sobre ese comportamiento — una distinción que vale la pena hacer explícita en clase, porque es fácil que el valor de esta semana se perciba como 'menos tangible' que construir un endpoint nuevo, cuando en realidad es la disciplina que hace sostenible construir endpoints nuevos, semana tras semana, sin que el sistema se vuelva progresivamente más frágil.

Referencias

- Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum.
- Pact Foundation. (2024). Pact Documentation. https://docs.pact.io
- Meszaros, G. (2007). xUnit Test Patterns: Refactoring Test Code.
- Jest. (2024). Jest Documentation. https://jestjs.io/docs/getting-started
- Postman. (2024). Newman Documentation. https://learning.postman.com/docs/collections/using-newman-cli/
- GitHub. (2024). GitHub Actions Documentation. https://docs.github.com/actions

Autoevaluación

1. Explica por qué invertir la pirámide de pruebas suele terminar en una suite lenta y frágil.

2. ¿Por qué el contract testing con Pact no requiere levantar el servicio proveedor real durante la prueba del consumidor?

3. Describe una situación del ecosistema del curso donde un microservicio actúe como consumidor en una relación y proveedor en otra.

4. Da un ejemplo propio donde usarías un stub, y otro donde necesitarías un spy en vez de un stub.

5. ¿Por qué la verificación de contrato suele ubicarse como un job separado, justo antes del despliegue?

6. ¿Qué le dirías a un compañero que afirma que 100% de cobertura de pruebas significa que su código no tiene bugs?

7. Explica la diferencia entre que un mutante 'sobreviva' y que 'sea eliminado' en mutation testing, y qué te dice cada resultado sobre tu suite.

8. ¿Por qué actualizar un snapshot con --updateSnapshot sin revisar el diff puede ser tan riesgoso como aceptar sin revisar un cambio de esquema en la Semana 7?

9. En el pipeline de dos workflows (consumidor y proveedor) de la sección 9.5, explica por qué ninguno de los dos equipos necesita acceso al código fuente del otro para que el mecanismo funcione.

10. Diseña, en un párrafo, un escenario propio (fuera del dominio de Productos) donde aplicarías la tabla de 'Señales de que a tu proyecto le falta un tipo de prueba', identificando el síntoma y el nivel de prueba que lo resolvería.